

Advanced Cryptography

Elisabeth Oswald (module lead), Sam Collins (teaching assistant)

Motivation: examples for MPC

THE CONTENT OF THIS MODULE

This module covers advanced topics in cryptography: this year with an emphasis on **secret sharing** and its application in the area of secure multiparty computation, and leakage resilient implementations.

If time permits, we will also look into threshold encryption and signatures, garbled circuits, as well as efficient zero-knowledge proofs via MPC in the head.

Introductory modules in cryptography tend to cover cryptographic techniques that are applied to “data at rest” and “data in transit”. **In this module we look at cryptographic techniques for “data in computation”.**

SECURING COMPUTATION

Multiparty computation: rather than just storing or sending data between multiple parties, we also wish to do some computation on data (and this may include key material).

Situations in practice include:

Parties might not trust each other: they may not wish to reveal their inputs to each other, but they wish to do some joint computation.

Parties might not trust a third party to compute on their data.

Interestingly: encryption using a secret key on an embedded system leaks information about the secret; thus one has to “split up” the secret to avoid such leakage during computation.

PRINCIPLES

Homomorphic encryption: the encryption function Enc has the property that $Enc(x_1 \cdot x_2 \cdot \dots \cdot x_n) = Enc(x_1) \star Enc(x_2) \star \dots \star Enc(x_n)$.

The idea is then that the distrusting parties (who each own data x_i), can compute $Enc(x_1 \cdot x_2 \cdot \dots \cdot x_n)$ without having to reveal their x_i values.

Secret sharing: there is a secret value x is represented as a tuple $(x_1, \dots, x_n)$, so that each party gets at least one share. Then one has to find ways to compute interesting functions using the shares only.

Garbled circuits: a special technique for 2party computation, based on representing interesting functions as Boolean circuits, which are then “garbled.”

We will deal with secret sharing techniques in this course, and take a very brief look at garbled circuits at the end.

EXAMPLE: COMPUTING AN AVERAGE OVER SECRET DATA

Suppose that we have n different people, each with a secret salary s_i . What we wish to compute $\frac{1}{n} \sum_{i=1}^n s_i$.

“Obvious” solution to this problem is to kind of “encrypt” s_i via $c_i = s_i + k_1 + k_2 + \dots + k_{n_1}$ (k_i are randomly chosen). This looks like the one-time pad with multiple keys, so is **perfectly secret**.

Now let's play a little bit with this idea: suppose that I choose n values to represent each salary $s_i = s_i^1 + s_i^2 + \dots + s_i^n$. Evidently the sum over all s_i can be computed by summing over all s_i^j .

EXAMPLE: COMPUTING AN AVERAGE OVER SECRET DATA, CONT.

Perhaps the next idea becomes now somewhat obvious: whilst nobody is willing to reveal s_i , it is safe to reveal all but one of the s_i^j to different parties:

$$\begin{array}{cccccc} P_1 & P_2 & \cdot & \cdot & \cdot & P_n \\ s_1^1 & s_1^2 & \cdot & \cdot & \cdot & s_1^n \\ s_2^1 & s_2^2 & \cdot & \cdot & \cdot & s_2^n \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ s_n^1 & s_n^2 & \cdot & \cdot & \cdot & s_n^n \end{array}$$

Now each party can compute part of the overall sum, without learning anything about the s_i : P_i can compute $\sum_{j=1}^n s_j^i$.

With each partial sum, the overall sum can also be computed: each party simply passes on their sum to the other parties.

EXAMPLE: MITIGATING INFORMATION LEAKAGE

Let us assume that an adversary can observe the power consumption of values being loaded in a processor: i.e. if s is fetched from memory, the adversary can observe $f(s)$.

If $f()$ is one-to-one, then the adversary can deduce s from observing $f(s)$.

If $f()$ is injective, then the adversary cannot learn s precisely, but they can learn the pre-image of $f(s)$.

Concretely: suppose that a device leaks the Hamming weight (HW) of data values, and that a data word is always 8 bits. So if the value 3 is manipulated, the device leaks $HW(3) = 2$. If an adversary thus sees the leakage 2, they know that the intermediate value was either $\{3, 5, 6, 9, 10, 12, 17, \dots\}$.

EXAMPLE: MITIGATING INFORMATION LEAKAGE, CONT.

Suppose that instead of manipulating the value 3, we manipulate 75 and 72.

Why? Check out the binary representations and observe that $3 = 75 \oplus 72$ ($\oplus$ equals addition modulo 2).

Now the observable leakages would be 2 and 4. If the adversary only gets one of the two observations, they learn nothing about s .

But wouldn't in practice they get leakage for both shares? Yes, and then the security argument gets more complicated.

MPC vs FHE

FHE: (fully homomorphic encryption), aka “computing on encrypted data”; main use case is outsourcing of private data.

MPC: (multi party computation); important use case is key sharing (example DNSSEC root keys are not known to any single party), but also privacy friendly aggregation of data.

Both techniques are a cornerstone of modern **privacy enhancing technologies** (PETs).

Secret sharing based MPC is a fundamental technique deployed to secure implementations (hardware and software) against side channel attacks.

MORE PRACTICE . . .

- ▶ Denmark: auction and bidding platform for sugar beet contracts
- ▶ Estonia: deploy Sharemind framework to analyse student/study related data
- ▶ US: Boston analyse salary data using MPC aggregation
- ▶ Worldwide: instead of hardware security modules, Unbound sells MPC based secure key storage in software
- ▶ Worldwide: MPC based bitcoin wallets
- ▶ Microsoft has developed ExPC which is an MPC library and provided an interface to secure Tensorflow
- ▶ There is an MPC enhanced version of Ethereum

All big players invest heavily in libraries to support MPC (and FHE as well as other PETs), deployments widen, this is a good field for “applied cryptographers”.